

GUIA DE ESTUDO

Calculadora Científica em Python + Tkinter

Guia completo para explicar cada linha do código ao professor

Desenvolvimento de Software — Python 3 | Tkinter | 2026

1. Visão Geral do Programa

Este programa é uma Calculadora Científica feita em Python usando a biblioteca Tkinter para criar a interface gráfica (janela, botões, display). O código é dividido em quatro partes principais:

- Configuração da janela e cores
- Criação do display (tela onde aparecem os números)
- Funções de lógica (o que acontece ao clicar em cada botão)
- Criação dos botões e layout

2. Bibliotecas Importadas

```
from tkinter import *  
import math
```

from tkinter import *

Importa TUDO da biblioteca tkinter de uma vez. O tkinter é a biblioteca padrão do Python para criar interfaces gráficas — janelas, botões, caixas de texto etc. O asterisco (*) evita ter que escrever 'tkinter.Button', permitindo escrever só 'Button'.

import math

Importa a biblioteca matemática do Python. É ela que fornece funções como `math.sin()`, `math.cos()`, `math.log()`, `math.pi`, `math.sqrt()` etc., usadas nos cálculos científicos da calculadora.

 **Dica para a prova**

Se o professor perguntar 'por que usar `from tkinter import *` ao invés de `import tkinter`?', a resposta é: para escrever menos código. Em vez de `tkinter.Button()`, escrevemos só `Button()`.

3. Definição de Cores

```
cor1 = "#3b3b3b" # Preto
cor2 = "#ffffff" # Branco
cor3 = "#38576b" # Azul (display)
cor4 = "#7d7c7c" # Cinza (botões normais)
cor5 = "#FFAB40" # Laranja (operadores / =)
cor6 = "#2a6496" # Azul escuro (botões científicos)
cor7 = "#5a5a5a" # Cinza médio
```

São variáveis simples que guardam códigos hexadecimais de cor — o mesmo formato usado em HTML/CSS. O '#' indica que é uma cor hexadecimal. Em vez de repetir '#3b3b3b' em todo o código, usamos o nome `cor1`, tornando o código mais fácil de ler e alterar.

Uso das cores: `cor1` = fundo da janela | `cor3` = fundo do display | `cor4` = botões cinza | `cor5` = botões laranja (operadores) | `cor6` = botões azuis (científicos)

4. Criação da Janela Principal

```
janela = Tk()
janela.title("Calculadora Científica")
janela.geometry("320x500")
janela.config(bg=cor1)
janela.resizable(False, False)
```

`janela = Tk()`

Cria a janela principal do programa. `Tk()` é a função do tkinter que abre uma janela em branco. Tudo que aparece na tela é filho dessa janela.

`janela.title("Calculadora Científica")`

Define o texto que aparece na barra do topo da janela (a barra azul com o nome do programa).

`janela.geometry("320x500")`

Define o tamanho da janela: 320 pixels de largura por 500 pixels de altura. O formato é sempre 'LarguraxAltura'.

`janela.config(bg=cor1)`

`config()` configura propriedades do widget. Aqui define a cor de fundo (`bg` = background) da janela como `cor1` (preto #3b3b3b).

`janela.resizable(False, False)`

Impede que o usuário redimensione a janela arrastando as bordas. O primeiro False bloqueia a largura, o segundo bloqueia a altura.

💡 Dica para a prova

Se o professor perguntar o que aconteceria se resizable fosse True: a janela poderia ser esticada e os botões ficariam desalinhados, pois o layout foi feito para tamanho fixo.

5. Variáveis de Estado

```
expressao    = ""
modo_graus   = True
segundo_mod0 = False
```

Variáveis de estado guardam informações sobre o que está acontecendo no programa enquanto ele roda.

expressao = ""

Começa vazia. Vai acumulando o que o usuário digita. Ex: se o usuário clica 7, depois *, depois 2, a variável vira "7*2". Quando o usuário clica =, o programa calcula essa string.

modo_graus = True

Controla se os cálculos trigonométricos usam graus (True) ou radianos (False). O botão DEG/RAD alterna esse valor.

segundo_mod0 = False

Controla se o botão '2nd' está ativado. Quando True, os botões sin/cos/tan viram asin/acos/atan (funções inversas).

💡 Dica para a prova

Essas três variáveis são acessadas e modificadas por várias funções usando a palavra-chave 'global'. Sem global, a função criaria uma cópia local e não mudaria o valor original.

6. O Display (Tela da Calculadora)

```
frame_tela = Frame(janela, width=320, height=80, bg=cor3)
frame_tela.grid(row=0, column=0, sticky="ew")
frame_tela.grid_propagate(False)

display_expr = StringVar(value="")
display_res  = StringVar(value="0")

lbl_expr = Label(frame_tela, textvariable=display_expr,
                  font=("Courier", 11), bg=cor3, fg="#aac8d8", anchor="e")
lbl_expr.place(x=0, y=6, width=314, height=18)

lbl_res = Label(frame_tela, textvariable=display_res,
                 font=("Courier", 28, "bold"), bg=cor3, fg=cor2, anchor="e")
lbl_res.place(x=0, y=28, width=314, height=46)
```

Frame — o container do display

Frame é uma caixa invisível que organiza outros widgets dentro dela. Aqui cria a área azul (cor3) do display no topo da janela.

`frame_tela.grid(row=0, column=0, sticky='ew')`

`grid()` posiciona o frame na grade da janela: linha 0, coluna 0. `sticky='ew'` estica o widget de leste a oeste (da esquerda até a direita).

`frame_tela.grid_propagate(False)`

Por padrão, um Frame encolhe para caber seus filhos. `grid_propagate(False)` desativa isso, forçando o Frame a manter o tamanho definido (320x80 pixels).

StringVar — variável especial do tkinter

StringVar é uma variável especial que, quando alterada via `.set()`, atualiza automaticamente qualquer Label que esteja conectada a ela. É a ponte entre o código Python e o que aparece na tela.

- `display_expr` = mostra a expressão que o usuário está digitando (linha de cima, menor)
- `display_res` = mostra o resultado ou o número atual (linha de baixo, grande)

Label — texto na tela

Label exibe texto. `textvariable=display_expr` conecta o Label à StringVar: quando a variável muda, o texto na tela muda automaticamente.

`lbl_expr.place(x=0, y=6, width=314, height=18)`

`place()` posiciona o widget com coordenadas absolutas em pixels. Diferente do `grid()`, que usa linhas e colunas, o `place()` diz exatamente onde colocar e qual tamanho ter.

O display tem DOIS labels: `lbl_expr` (cima, mostra '7*2 =') e `lbl_res` (baixo, maior, mostra '14'). Esse padrão é igual ao da calculadora do Windows.

7. Função `ajustar_fonte()`

```
def ajustar_fonte(texto):
    n = len(str(texto))
    if n <= 9: tam = 28
    elif n <= 13: tam = 22
    elif n <= 18: tam = 16
    elif n <= 24: tam = 12
    else: tam = 9
    lbl_res.config(font=("Courier", tam, "bold"))
```

`def` define uma função — um bloco de código que só roda quando chamado.

`len(str(texto))` conta quantos caracteres tem o texto. Quanto mais caracteres, menor a fonte fica — igual à calculadora do Windows. Isso evita que números longos sejam cortados.

`lbl_res.config()` altera a propriedade `font` do Label depois que ele já foi criado, sem precisar recriar o widget.

elif significa 'else if': é testado só se o if anterior foi falso. else é o caso final, quando nenhuma condição anterior foi verdadeira.

Tamanhos: até 9 caracteres = fonte 28 | até 13 = fonte 22 | até 18 = fonte 16 | até 24 = fonte 12 | mais de 24 = fonte 9

8. Funções de Lógica

8.1 adicionar()

```
MAX_CHARS = 32

def adicionar(simbolo):
    global expressao
    if len(expressao) >= MAX_CHARS:
        return
    expressao += str(simbolo)
    display_expr.set(expressao)
    display_res.set(expressao)
    ajustar_fonte(expressao)
```

É chamada toda vez que o usuário clica em um número ou operador.

- MAX_CHARS = 32: limite de 32 caracteres. Após isso, novos cliques são ignorados.
- global expressao: diz ao Python para usar a variável expressao do escopo global, não criar uma nova local.
- if len(expressao) >= MAX_CHARS: return: se já atingiu 32 chars, sai da função sem fazer nada.
- expressao += str(simbolo): junta o novo símbolo ao final. Ex: '7' + '*' + '2' = '7*2'.
- display_expr.set() e display_res.set(): atualizam os dois Labels do display.
- ajustar_fonte(): redimensiona a fonte conforme o tamanho do texto.

8.2 limpar()

```
def limpar():
    global expressao
    expressao = ""
    display_expr.set("")
    display_res.set("0")
    ajustar_fonte("0")
```

Chamada pelo botão C. Zera a expressão e volta o display para '0'. É a operação mais simples do programa.

8.3 apagar()

```
def apagar():
    global expressao
    expressao = expressao[:-1]
```

```
display_expr.set(expressao)
val = expressao if expressao else "0"
display_res.set(val)
ajustar_fonte(val)
```

Chamada pelo botão ✕. Remove o último caractere da expressão.

`expressao[:-1]` é uma fatia (slice) de string: pega todos os caracteres MENOS o último. Ex: `'123'[:-1] = '12'`.

`expressao if expressao else '0'`: se a expressão ficar vazia após apagar, mostra '0' no display em vez de deixar em branco.

8.4 calcular()

```
def calcular():
    global expressao
    try:
        resultado = eval(expressao)
        if isinstance(resultado, float) and resultado.is_integer():
            resultado = int(resultado)
        s = str(resultado)
        display_expr.set(expressao + " =")
        display_res.set(s)
        ajustar_fonte(s)
        expressao = s
    except ZeroDivisionError:
        display_res.set("Div/0!")
        ajustar_fonte("Div/0!")
        expressao = ""
    except Exception:
        display_res.set("Erro")
        ajustar_fonte("Erro")
        expressao = ""
```

É a função mais importante. Chamada quando o usuário clica em = ou pressiona Enter.

- `eval(expressao)`: recebe a string `'7*2+3'` e executa como código Python, retornando 17.
- `isinstance(resultado, float) and resultado.is_integer()`: se o resultado for 14.0, converte para 14 (sem a casa decimal).
- `try/except`: tenta executar o bloco. Se der erro, cai no except sem travar o programa.
- `ZeroDivisionError`: capturado separadamente, mostra 'Div/0!' ao invés de 'Erro'.
- `except Exception`: captura qualquer outro erro (expressão inválida, raiz de negativo etc.).

💡 Dica para a prova

Se o professor perguntar 'o que é `eval()`?': é uma função nativa do Python que executa uma string como código. `eval('2+3')` retorna 5, `eval('2**10')` retorna 1024.

9. Funções de Conversão de Ângulo

```
def _angulo(x):
    return math.radians(x) if modo_graus else x

def _de_rad(x):
    return math.degrees(x) if modo_graus else x
```

A biblioteca math sempre trabalha com radianos. Essas duas funções fazem a conversão automática dependendo do modo atual.

_angulo(x)

Antes de chamar `math.sin()`, `math.cos()` ou `math.tan()`, o valor passa por aqui. Se `modo_graus` for `True`, converte de graus para radianos (`math.radians`). Se for `False`, usa o valor como está.

_de_rad(x)

Usada nas funções inversas (`asin`, `acos`, `atan`). O resultado delas é sempre em radianos, então converte para graus se o `modo_graus` for `True`.

Operador ternário: valor_se_true if condicao else valor_se_false

É uma forma compacta de escrever `if/else` em uma linha. '`math.radians(x) if modo_graus else x`' equivale a: se `modo_graus` for `True`, retorna `math.radians(x)`, senão retorna `x`.

10. Função aplicar_func()

```
def aplicar_func(func_nome):
    global expressao
    try:
        val = float(eval(expressao)) if expressao else 0.0
        if func_nome == "sin": r = math.sin(_angulo(val))
        elif func_nome == "cos": r = math.cos(_angulo(val))
        elif func_nome == "tan": r = math.tan(_angulo(val))
        # ... demais funções ...
        if isinstance(r, float) and r.is_integer():
            r = int(r)
        s = str(round(float(r), 10))
        if '.' in s:
            s = s.rstrip('0').rstrip('.')
        display_expr.set(f"{func_nome}({expressao}) =")
        display_res.set(s)
        ajustar_fonte(s)
        expressao = s
    except Exception:
        display_res.set("Erro")
        expressao = ""
```

Chamada pelos botões científicos: `sin`, `cos`, `tan`, `log`, `ln`, `sqrt`, `x²`, `n!` etc.

- `float(eval(expressao))`: converte a expressão atual em número decimal para passar às funções `math`.
- `if expressao else 0.0`: se a expressão estiver vazia, usa 0 como valor.

- `round(float(r), 10)`: arredonda para 10 casas decimais, evitando resultados como 0.9999999999.
- `s.rstrip('0').rstrip('.')`: remove zeros desnecessários. Ex: '3.50000' vira '3.5'. '3.0' vira '3'.
- `f'{func_nome}({expressao}) ='`: f-string — forma de inserir variáveis dentro de texto. Ex: `'sin(30) ='`.

💡 Dica para a prova

Todas as funções científicas passam por `aplicar_func()`. O parâmetro `func_nome` é uma string como 'sin', 'log', 'sqrt' etc. O `if/elif` decide qual cálculo fazer.

11. Funções de Alternância (Toggle)

```
def toggle_graus():
    global modo_graus
    modo_graus = not modo_graus
    btn_deg.config(text="DEG" if modo_graus else "RAD",
                   bg=cor6 if modo_graus else "#c0392b")
```

`not` inverte um valor booleano: `not True = False`, `not False = True`. Cada clique alterna o modo.

`btn_deg.config()` altera o texto e a cor do botão: DEG (azul) ou RAD (vermelho), refletindo o estado atual.

```
def toggle_2nd():
    global segundo_mod0
    segundo_mod0 = not segundo_mod0
    if segundo_mod0:
        btn_sin.config(text="asin")
        btn_cos.config(text="acos")
        btn_tan.config(text="atan")
        btn_2nd.config(bg=cor5)          # laranja = ativo
    else:
        btn_sin.config(text="sin")
        btn_cos.config(text="cos")
        btn_tan.config(text="tan")
        btn_2nd.config(bg=cor6)          # azul = inativo
```

Quando `segundo_mod0 = True`, os três botões de trigonometria mudam de texto: `sin` vira `asin`, `cos` vira `acos`, `tan` vira `atan`. O botão 2nd fica laranja para indicar que está ativo.

`btn_sin`, `btn_cos` e `btn_tan` precisam ser salvos como variáveis (e não descartados) justamente para que `.config()` funcione aqui.

12. Suporte ao Teclado

```
def tecla(event):
    k = event.keysym
    char = event.char
    if char in "0123456789":      adicionar(char)
    elif char == ".":            adicionar(".")
    elif char == "+":            adicionar("+")
    # ... demais teclas ...
    elif k == "KP_0":            adicionar("0")
    elif k == "KP_1":            adicionar("1")
    # ... demais teclas do pad numérico ...
    elif k in ("Return", "KP_Enter"): calcular()
    elif k == "BackSpace":       apagar()
    elif k == "Escape":          limpar()

janela.bind("<Key>", tecla)
```

Quando qualquer tecla é pressionada, o tkinter cria um objeto 'event' com informações sobre ela.

- event.char: o caractere digitado ('7', '+', '.').
- event.keysym: o nome simbólico da tecla ('KP_7' para o 7 do teclado numérico, 'Return' para Enter, 'BackSpace' para apagar).
- KP_ são as teclas do teclado numérico (KeyPad). 'KP_7' é o 7 do pad numérico, diferente do '7' da linha de cima.

janela.bind('<Key>', tecla)

bind() conecta um evento a uma função. Aqui: 'quando qualquer tecla for pressionada na janela, chame a função tecla'. O '<Key>' captura qualquer tecla do teclado.

Dica para a prova

Atalhos: Enter ou KP_Enter = calcular | Backspace = apagar | Escape = limpar | Números e operadores (+, -, *, /) funcionam tanto no teclado principal quanto no numérico.

13. Construtor de Botões — função btn()

```
def btn(parent, texto, linha, col, cmd,
        cspan=1, rspan=1, bg=cor4, fg=cor2):
    b = Button(parent, text=texto, bg=bg, fg=fg,
                font=("Ivy", 11, "bold"),
                relief=FLAT, overrelief=RIDGE,
                command=cmd, cursor='hand2',
                bd=1, highlightthickness=0)
    b.grid(row=linha, column=col, columnspan=cspan, rowspan=rspan,
           padx=1, pady=1, sticky='nsew')
    return b
```

Em vez de repetir 15 linhas para cada botão, criamos uma função auxiliar que fabrica botões com as configurações padrão.

Parâmetros da função

- parent: onde o botão vai ficar (frame_corpo)

- texto: o texto escrito no botão ('7', '+', 'sin' etc.)
- linha, col: posição na grade (grid)
- cmd: a função chamada ao clicar (lambda: adicionar('7'), calcular, limpar etc.)
- cspan=1, rspan=1: quantas colunas/linhas o botão ocupa (padrão = 1 cada)
- bg=cor4, fg=cor2: cor de fundo e cor do texto. Têm valores padrão mas podem ser sobrescritos

Propriedades do Button

- relief=FLAT: botão sem relevo (visual limpo)
- overrelief=RIDGE: aparece um relevo quando o mouse passa por cima
- command=cmd: a função que será chamada ao clicar
- cursor='hand2': muda o cursor para uma mãozinha ao passar no botão

b.grid() — posicionamento na grade

- padx=1, pady=1: 1 pixel de espaço entre os botões (cria as linhas da grade)
- sticky='nsew': estica o botão para preencher toda a célula (norte, sul, leste, oeste)

14. Frame dos Botões e Grade

```
frame_corpo = Frame(janela, bg=cor1, width=320, height=420)
frame_corpo.grid(row=1, column=0, sticky='nsew')
frame_corpo.grid_propagate(False)

for i in range(7):
    frame_corpo.rowconfigure(i, weight=1)
for j in range(5):
    frame_corpo.columnconfigure(j, weight=1)
```

O frame_corpo ocupa os 420px restantes da janela (500 total - 80 do display = 420).

for i in range(7): frame_corpo.rowconfigure(i, weight=1)

for repete o bloco para cada valor em range(7) = [0,1,2,3,4,5,6].

rowconfigure/columnconfigure configura cada linha e coluna da grade. weight=1 significa que todas têm o mesmo peso, dividindo o espaço igualmente — todos os botões ficam do mesmo tamanho.

A grade tem 7 linhas × 5 colunas = 35 posições. Os botões científicos ficam na coluna 0 (azul), os numéricos nas colunas 1, 2 e 3 (cinza) e os operadores na coluna 4 (laranja).

15. Layout dos Botões — Linha por Linha

```
# Linha 0: 2nd | DEG | π | C | ☒
btn_2nd = btn(frame_corpo, '2nd', 0, 0, toggle_2nd, bg=cor6)
btn_deg = btn(frame_corpo, 'DEG', 0, 1, toggle_graus, bg=cor6)
btn(frame_corpo, 'π', 0, 2, lambda: inserir_constante(str(math.pi)), bg=cor6)
btn(frame_corpo, 'C', 0, 3, limpar, bg=cor4)
btn(frame_corpo, '☒', 0, 4, apagar, bg='#c0392b')
```

Cada chamada de `btn()` cria um botão na posição especificada.

- `btn_2nd` e `btn_deg` são salvos em variáveis porque `toggle_2nd()` e `toggle_graus()` precisam chamar `.config()` neles depois.
- `lambda: inserir_constante(str(math.pi))`: uma função anônima criada na hora. Quando clicado, chama `inserir_constante` passando o valor de π (3.14159...).
- `bg='#c0392b'`: o botão  tem cor vermelha diretamente, sem usar uma variável de cor.

```
# Linha 2: sin | cos | tan | n! | ÷
btn_sin = btn(frame_corpo, 'sin', 2, 0, lambda: aplicar_func(trig()),
bg=cor6)
btn_cos = btn(frame_corpo, 'cos', 2, 1, lambda: aplicar_func(trig_cos()),
bg=cor6)
btn_tan = btn(frame_corpo, 'tan', 2, 2, lambda: aplicar_func(trig_tan()),
bg=cor6)
```

`btn_sin`, `btn_cos` e `btn_tan` são salvos em variáveis para que `toggle_2nd()` possa mudar o texto deles para 'asin', 'acos' e 'atan'.

`lambda: aplicar_func(trig())`: ao clicar, chama `trig()` que retorna 'sin' ou 'asin' dependendo do modo 2nd, e passa para `aplicar_func()`.

16. `janela.mainloop()`

```
janela.mainloop()
```

É a ÚLTIMA linha do programa e uma das mais importantes. `mainloop()` inicia o loop de eventos do tkinter — mantém a janela aberta e fica esperando ações do usuário (cliques, teclas, fechar janela).

Sem essa linha, o programa criaria a janela e a fecharia imediatamente, pois o Python terminaria a execução do script.

Dica para a prova

Se o professor perguntar 'o que é um loop de eventos?': é um ciclo que fica rodando infinitamente, verificando se o usuário fez alguma coisa (cliquou em um botão, pressionou uma tecla) e chamando a função correspondente. Ele só para quando o usuário fecha a janela.

17. Resumão — O que cada parte faz

Cole este resumo na memória! O professor pode perguntar sobre qualquer linha.

Elemento	O que faz
<code>Tk()</code>	Cria a janela principal do programa
<code>geometry('320x500')</code>	Define o tamanho: 320px de largura por 500px de altura
<code>resizable(False, False)</code>	Impede redimensionamento da janela
<code>Frame</code>	Container invisível que organiza outros widgets
<code>grid()</code>	Posiciona o widget em linha/coluna na grade
<code>grid_propagate(False)</code>	Mantém o tamanho fixo do Frame
<code>StringVar</code>	Variável ligada ao display — muda a tela automaticamente
<code>Label</code>	Exibe texto na tela (os dois textos do display)
<code>place()</code>	Posiciona widget com coordenadas exatas em pixels
<code>Button</code>	Cria um botão clicável
<code>command=</code>	Define qual função chamar ao clicar no botão
<code>bind('<Key>', tecla)</code>	Captura teclas do teclado físico
<code>eval()</code>	Calcula uma expressão em string ('7*2+3' → 17)
<code>global</code>	Permite que a função modifique a variável global
<code>try/except</code>	Tenta o código; se der erro, cai no except
<code>not</code>	Inverte booleano: not True = False
<code>string[:-1]</code>	Remove o último caractere (usado no apagar)
<code>.config()</code>	Altera propriedade de um widget já criado
<code>mainloop()</code>	Mantém a janela aberta, aguardando ações do usuário
<code>lambda:</code>	Cria uma função anônima pequena na hora
<code>weight=1</code>	Divide o espaço da grade igualmente entre linhas/colunas